

Data Speaks!! — Project Abstract & Preprocessing Methodology

Abstract

This project explores the application of machine learning and deep learning architectures on the benchmark Large Movie Review Dataset (IMDB). The dataset comprises 100,000 highly polar movie reviews, divided into binary classification, continuous regression, and unsupervised subsets. The primary objective is to develop models capable of extracting underlying sentiment and predicting precise user ratings on a 1-10 scale. To handle the inherently noisy, unstructured nature of web-scraped natural language, a comprehensive and robust data extraction and preprocessing pipeline was engineered. This document details the exact preprocessing methodologies utilized before feeding the data into baseline statistical models (Naive Bayes, Logistic Regression), state-of-the-art Deep Learning models (LSTM, BERT), and Dimensionality Reduction algorithms (Truncated SVD).

1. Data Extraction & Label Parsing

The raw IMDB dataset is distributed as thousands of individual text files grouped in directories. Before any NLP preprocessing could begin, the data had to be successfully parsed into tabular CSV formats.

Binary Classification: Labels were extracted directly from the directory structure (e.g., reviews in the 'pos/' folder were assigned a label of 1, while reviews in the 'neg/' folder were assigned a label of 0).

Continuous Regression: To build the regression dataset, the precise 1-10 user rating was extracted directly from the text file metadata via string manipulation (parsing the `[id]\_[rating].txt` filename format).

2. Core Text Preprocessing Pipeline

Web-scraped data contains significant HTML artifacts. Upon analysis, over 50% of the raw reviews contained HTML line breaks. A universal cleaning pipeline was applied across all models:

HTML Unescaping & Tag Stripping: We utilized Regular Expressions (regex) to permanently remove rogue HTML elements, specifically substituting ` `

tags with empty spaces.**Punctuation Filtering:** For baseline models and WordCloud Exploratory Data Analysis, special characters were stripped using the regex `[^\a-zA-Z\s]`, keeping only standard alphabetical characters.

Whitespace Normalization & Lowercasing: Multiple spaces were collapsed into single spaces to ensure token consistency, and all text was converted to lowercase to dramatically reduce the sparse vocabulary size.

3. Model-Specific Vectorization & Tokenization

Following the core cleaning pipeline, the raw text was transformed into numerical representations tailored to the specific architecture of each model:

TF-IDF Vectorization (Logistic Regression, Naive Bayes, SVD): Text was tokenized into unigrams and bigrams, limited to the top 10,000 most frequent features. Term Frequency-Inverse Document Frequency (TF-IDF) scoring was applied to build a highly sparse numerical matrix, weighting words by their relative uniqueness.

Integer Tokenization & Padding (LSTM): Text was tokenized and passed through a custom Word2Idx vocabulary builder. Each word was mapped to a unique integer. Sequences were subsequently padded or truncated to a fixed sequence length of 200 tokens to ensure uniform matrix dimensions before being passed to a 128-dimensional Embedding Layer.

Sub-word Tokenization (BERT Classification & Regression): Deep learning transformers require specific token formatting. We utilized the pre-trained `bert-base-uncased` tokenizer, which applies sub-word (WordPiece) tokenization to handle unknown vocabulary dynamically. Sentences were padded to 512 tokens, and Attention Masks were generated to instruct the Transformer on which tokens represented

valid words versus padding.